Software Requirements Specification

Food Truck Order Management System

Living Document – Agile Approach

Team Sleepers

Hassan Yousef	13006567
Sara Adel	14003723
Hana Yasser	13003628
Mohamed Walid	13006513
Omar Hani	13007515
Abdelhamid ElSharnouby	13006294
Hanin Mohamed	13007010
Khaled Khaled	14001048

German International University (GIU)

Supervised by:

Dr. Iman Awaad
Eng. Amir Haytham

December 2, 2025

Contents

1 Introduction **1**
1.1 Purpose 1
1.2 Intended Audience 1
1.3 Scope 1
1.4 Overview 2

2 Product Vision and Scope **3**
2.1 Product Vision 3
2.2 In Scope 3
2.2.1 Customer Features 3
2.2.2 Vendor Features 3
2.2.3 Admin Features 4
2.3 Out of Scope 4

3 High-Level Architecture **5**
3.1 Database Schema: Entity Relationship Diagram 5

4 Functional Requirements **6**
4.1 User Stories 6
4.1.1 Food Truck Customer User Stories 6
4.1.2 Food Truck Service Provider User Stories 8
4.1.3 Food Truck Admin User Stories 9
4.2 Use Case Diagrams 10

5 Non-Functional Requirements **13**

6 Interface Requirements **14**

7 Constraints **15**
7.1 Technical Constraints 15
7.2 Operational Constraints 15
7.3 Resource Constraints 15
7.4 Regulatory and Policy Constraints 16
7.5 User and Usability Constraints 16
7.6 Design and Architectural Constraints 16

8 Future Scope of the Project **17**

Notes **18**

1 Introduction

1.1 Purpose

Problem Statement:

Students and staff at GIU face significant challenges during peak hours when ordering food from campus food trucks. Long waiting lines, unclear preparation times, and lack of order visibility create frustration and time loss, especially between classes when time is limited.

Solution:

The GIU Food Truck Order Management System is a comprehensive web-based application designed to enable the members of the GIU community to reserve menu items from specific food trucks on campus for pick-up within a given time window, thus alleviating the problem with long waiting times for food and beverage items during peak periods in between classes.

This system enables seamless menu browsing, order placement, and order management for food truck customers as well as providing a similarly intuitive, efficient, and scalable solution for food service providers.

1.2 Intended Audience

- **Project Team:** Hassan Yousef, Sara Adel, Hana Yasser, Mohamed Walid, Omar Hany, Abdelhamid ElSharnouby, Hanin Mohamed, Khaled Khaled.
- **Developers and Testers:** Project team members.
- **Product Owner:** Doctor Iman Awaad.
- **Instructors:** Dr. Iman Awaad, Eng. Amir Haytham and Eng. Ahmad Sherif

1.3 Scope

The Food-truck System is a website that helps students and staff at GIU order food from the food trucks on campus. People can open the menus of different food trucks to see what food and drinks are available. They can then choose the items they want and make their order online. The system lets them pick up their food in a specific time, so they don't have to wait in long lines. This makes it easier for everyone to get their food, especially during busy times between classes. The system also helps the food trucks manage orders better, making sure everything runs smoothly for both customers and service providers. In general, the goal is to save time and make ordering food on campus simple and comfortable for the entire community.

1.4 Overview

The Food Truck Order Management System is a simple website for ordering food from the food trucks on the GIU campus. It helps to reduce waiting in lines by letting users place orders before going to the truck.

Who uses it:

- **Students and Staff (Customers):** Browse trucks, view menus, select items, place an order, choose (or accept) a pickup time, and check the order status (received, preparing, ready) on the website.
- **Food Truck Staff:** See incoming orders, update each order's status, manage menu items (add or remove).
- **Administrator:** Add or remove food trucks and oversee basic platform usage.

How order tracking works: The page refreshes (or auto-checks) every short interval to show updated order status. It does not use push notifications or advanced real-time technology.

What it does NOT include: No delivery, no online payment, no mobile app, no live chat, no push notifications, no multi-language (English only), no external integrations.

Why provider hours are longer: Food truck staff have extra time (before and after customer hours) to prepare ingredients and clean up.

Main goal: Save time, make ordering clearer and easier, and help trucks manage orders better.

Other notes: Works in a normal web browser on phones, tablets, and laptops. Designed for up to about 500 users at once.

2 Product Vision and Scope

2.1 Product Vision

The **Food Truck Order Management System** envisions a seamless, digital-first campus dining experience that minimizes waiting time and optimizes operations for both customers and vendors. The system will serve as a centralized web platform connecting GIU students, staff, and food truck operators through a polling-based ordering process.

From a user perspective, the vision is to eliminate the inefficiency of physical queues by allowing users to browse menus, place customized orders, and schedule pickups in advance. For food truck operators, the platform provides a unified dashboard to manage incoming orders and track preparation progress. Administrators will be able to manage participating vendors, monitor overall platform activity, and ensure service quality across all trucks.

The long-term vision is to transform how on-campus food services operate — creating a scalable digital ecosystem that supports automated order tracking, predictive demand analytics, and an integrated payment experience. Ultimately, this system aims to enhance convenience, transparency, and satisfaction for the GIU community while providing actionable insights to vendors and administrators for continuous operational improvement.

2.2 In Scope

The system includes the following features as part of its core functionality, categorized by user role.

2.2.1 Customer Features

- **User Authentication:** Customers can register and log in using email and password.
- **Menu Browsing:** Users can browse a list of food trucks, then view each truck's menu of food items.
- **Online Ordering:** Customers can place orders and select pickup times.
- **Estimated Preparation Time:** The system displays estimated time before confirming an order.
- **Order Status Tracking:** Customers can track order progress (Pending, In Progress, Ready).
- **Busy Mode on menu:** Customers can see if a vendor is currently too busy to accept new orders.

2.2.2 Vendor Features

- **View and Manage Orders:** Vendors can view incoming orders and update their statuses.

- **Menu Management:** Vendors can add or remove menu items.
- **Activate Busy Mode:** Vendors can temporarily stop accepting new orders.

2.2.3 Admin Features

- **Food truck management:** Admin can add or remove food trucks.

2.3 Out of Scope

The following features are explicitly excluded from the current system scope:

- **Restaurant Rating:** Users cannot rate or review food trucks or individual menu items. No feedback mechanism or star rating system is included in this release.
- **Bestsellers Menu:** The system does not track or display popular items, trending dishes, or "most ordered" recommendations. No sales analytics or item popularity features are provided.
- **Mobile Application:** The system will only be accessible via web browsers; no Android or iOS apps are included in this release.
- **Online Payment Integration:** Users will not be able to pay online. All transactions will be handled offline or through existing university methods.
- **Delivery Services:** Food delivery is not supported. Customers must pick up their orders from the food trucks in person.
- **University Authentication System Integration:** The system does not connect to GIU's single sign-on or internal authentication infrastructure.
- **Push Notifications:** Real-time push notifications (e.g., mobile or browser alerts) are not supported in this release. Users must track their orders through the website interface.
- **Multiple Language Support:** The system will only support English in its initial release. Support for other languages is excluded from scope.
- **Live Chat or Messaging:** No communication feature (e.g., chat with vendors or support) is included.
- **Advanced Real-Time Communication:** The system will use simple periodic refresh or polling to check order updates. Advanced real-time communication technologies (WebSockets, Server-Sent Events) are not included in this version.
- **Third-Party Integrations:** No integration with external services (e.g., social media, third-party APIs) is planned.
- **Advanced Analytics:** The system will not include complex analytics or reporting features for vendors or administrators.

3 High-Level Architecture

- **Front End:** Modern HTML, CSS, JavaScript, and jQuery. Or JavaScript Framework
- **Back End:** RESTful API using Node.js
- **Database:** Relational (PostgreSQL)
- **Authentication:** Email/password authentication (no external OAuth)
- **Payment:** Currently, none (offline transactions only)
- **Hosting:** To be determined (university servers or cloud provider)

3.1 Database Schema: Entity Relationship Diagram

The system utilizes a normalized relational database schema designed for data integrity, scalability, and audit trail support.

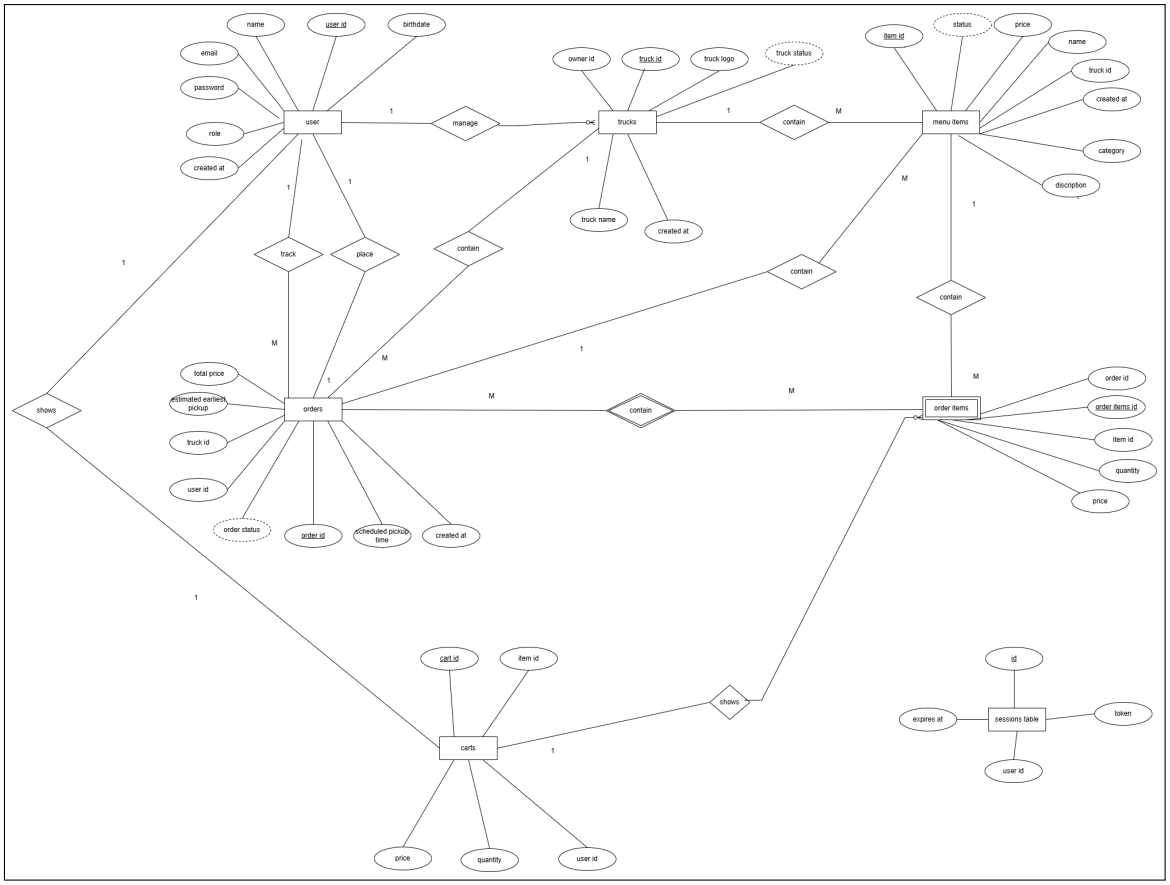

Figure 1: **Entity Relationship Diagram - Database Schema**

4 Functional Requirements

4.1 User Stories

4.1.1 Food Truck Customer User Stories

- As a customer, I want to schedule my lunch order in advance so that I can check the order status through the website and know when my order is ready for pickup.
- As a customer, I should be able to view all trucks that have truckStatus as "available" and orderStatus as "available" . The server should respond with an array of truck objects. The results should be sorted according to column truckId in table Trucks.
- As a customer, I should be able to place an order from my shopping cart with a scheduled pickup time. You must use the function getUser to retrieve the current customer id (userId).
- As a customer, I should be able to add any available menu item to my shopping cart. You need to insert a new cart record in your database table Carts. You must use the getUser function to retrieve the current customer id (userId). Before adding the menu item to cart, verify that all items in the cart belong to the same truck (truckId). If items are from different trucks, respond with error message "Cannot order from multiple trucks" . Otherwise, the server should respond with a message " item added to cart successfully" .
- As a customer, I should be able to view all items in my shopping cart based on my userId. You must use the function getUser to retrieve the current customer id (userId). The server should respond with an array of cartItem objects. This array should be sorted according to column cartId in table Carts.
- As a customer, I should be able to remove any item from my shopping cart. You need to delete a cart record using request params cartId. You must verify that the cart item belongs to the current user before deleting. The server should respond with a message " item removed from cart successfully" .

- As a customer, I should be able to edit the quantity of items in my shopping cart. You need to update a cart record using request params cartId. You must verify that the cart item belongs to the current user before updating. The server should respond with a message "cart updated successfully" .
- As a customer, I should be able to view all my past and current orders. You must use the function getUser to retrieve the current customer id (userId). The server should respond with an array of order objects including the truck name by joining with the Trucks table. The results should be sorted by orderId in descending order (most recent first).
- As a customer, I should be able to view the full details of a specific order including all items. You must verify that the order belongs to the current user before returning the details. The server should respond with an orderDetails object that includes order information and an array of items from the OrderItems table joined with MenuItems table.
- As a customer, I should be able to search and filter menu items by category for a specific truck. Only items with status "available" should be returned. The server should respond with an array of menuItem objects matching the specified category. The results should be sorted according to column itemId in table MenuItems.
- As a customer, I should be able to view all menu items for a specific truck where status is "available" . The server should respond with an array of menuItem objects. The results should be sorted according to column itemId in table MenuItems.

4.1.2 Food Truck Service Provider User Stories

- As a truck owner, I should be able to add items in to my menu. You need to insert a new menu item record in your MenuItems database table. You must use the default value of status column and createdAt column. The server should respond with a message that "menu item was created successfully".
- As a truck owner, I should be able to view all my menu items where status has a value of "available" . I am not allowed to view other truck owner's menu items. The server should respond with an array of menuItem objects. The results should be sorted according to column itemId in table MenuItems.
- As a truck owner, I should be able to view all information of a specific menu item using its itemId. I am not allowed to view menu items from other truck owners. The server should respond with a menuItem object.
- As a truck owner, I should be able to edit my menu items only. I am not allowed to edit other truck owner's menu items. You need to update a menu item record in your table MenuItems. You are allowed to edit name, price, category, and description only. The server should respond with a message "menu item updated successfully" .
- As a truck owner, I should be able to delete my menu items only. I am not allowed to delete other truck owner's menu items. You need to update a menu item record in your table MenuItems by setting its status to "unavailable" . The server should respond with a message "menu item deleted successfully" .

- As a truck owner, I should be able to view all orders placed for my truck. You must use the function getUser to retrieve the current truck owner's truckId. The server should respond with an array of truckOrder objects including the customer name by joining with the Users table. The results should be sorted by orderId in descending order (most recent first).
- As a truck owner, I should be able to update the status of orders for my own truck only. Valid values for orderStatus are "pending" , "preparing" , "ready" , "completed" , and "cancelled" . Before updating an order, the server must verify that the order belongs to the currently logged-in truck owner's truck. The estimatedEarliestPickup field may also be included in the update, but it is optional and should only be updated if provided. After successfully applying the update, the server should respond with the message: "order status updated successfully" .
- As a truck owner, I should be able to view the full details of a specific order including all items. You must verify that the order belongs to the current truck owner before returning the details. The server should respond with an orderDetails object that includes order information and an array of items from the OrderItems table joined with MenuItems table.
- As a truck owner, I should be able to toggle my truck's order availability. Valid orderStatus values are: "available" or "unavailable" . You must use the function getUser to retrieve the current truck owner's truckId. You need to update the orderStatus column in the Trucks table for your truck only. The server should respond with a message "truck order status updated successfully" .
- As a truck owner, I should be able to view my truck's information including current status settings. You must use the function getUser to retrieve the current truck owner's truckId. The server should respond with a truck object containing all information about the truck owner's truck.

4.1.3 Food Truck Admin User Stories

- As an administrator, I add or remove food trucks.

4.2 Use Case Diagrams

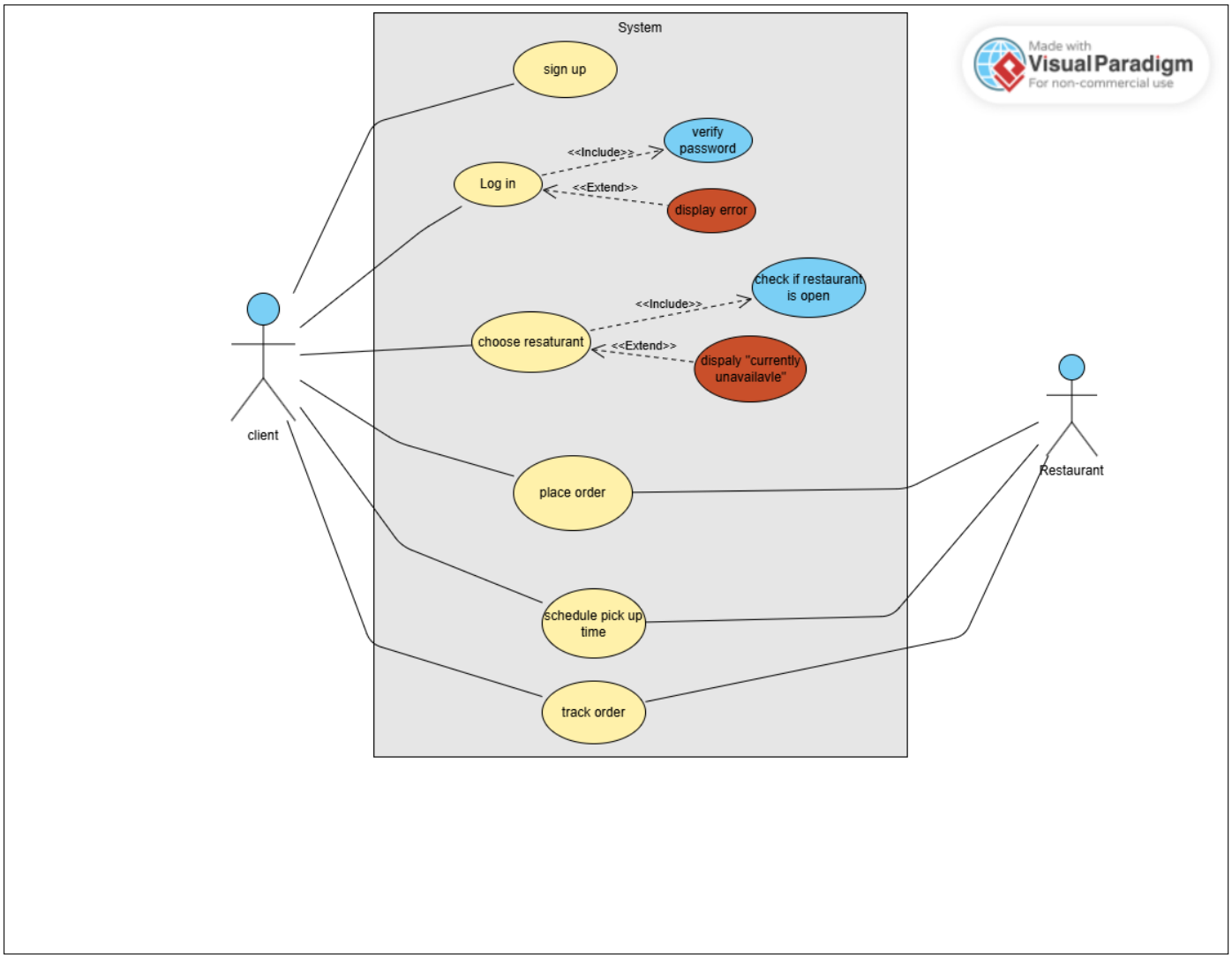

Figure 2: **Client Use Case Diagram**

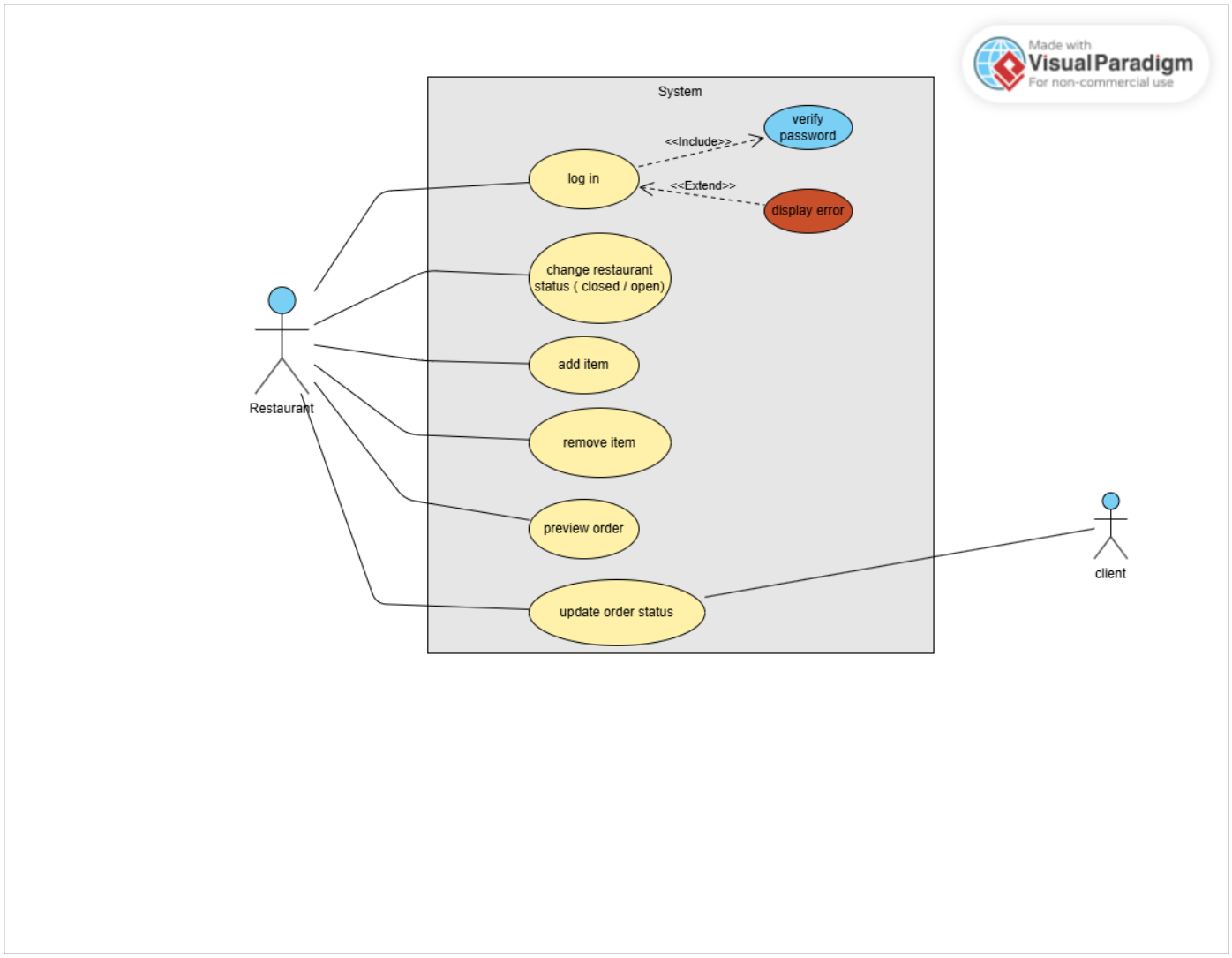

Figure 3: **Restaurant Use Case Diagram**

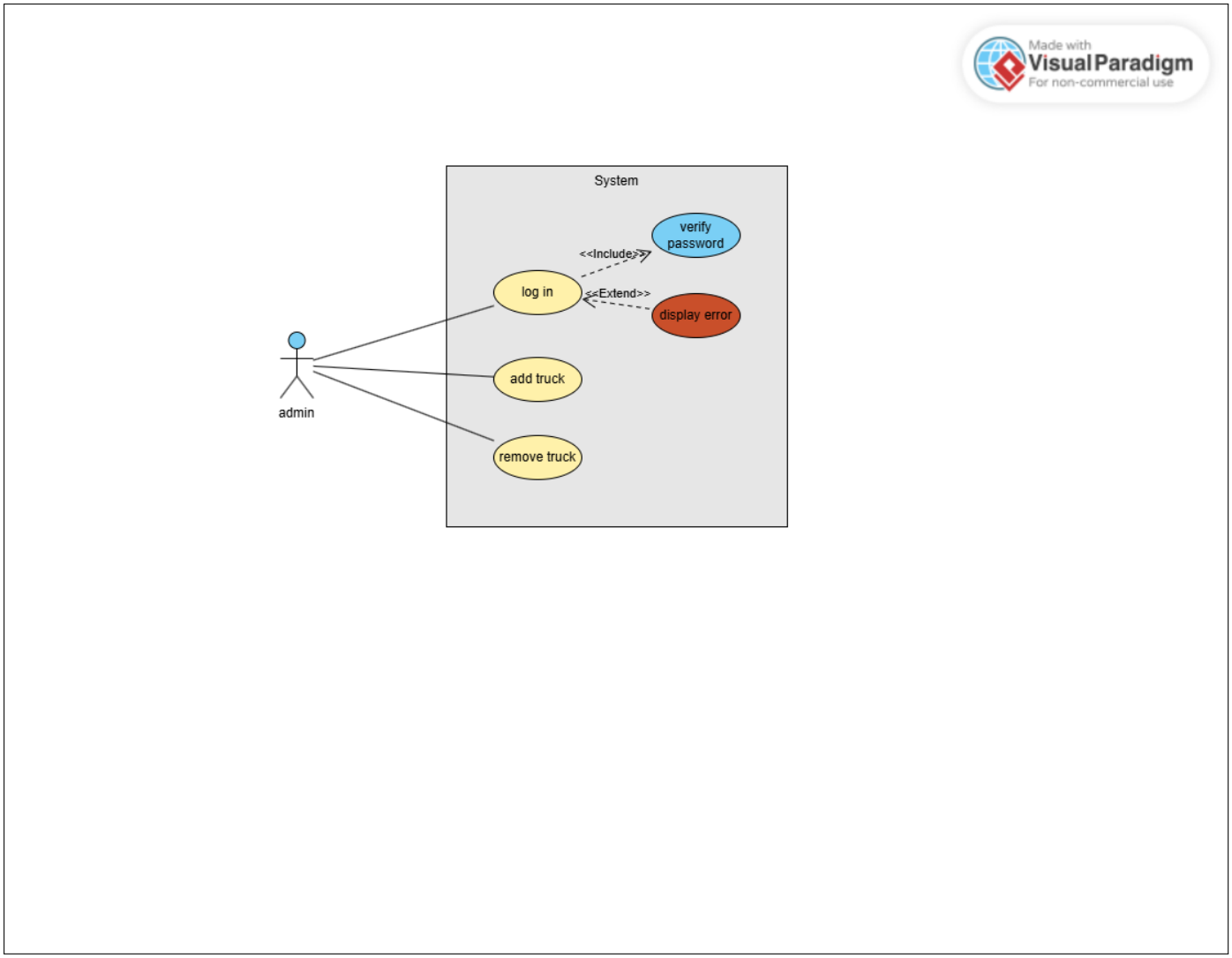

Figure 4: **Admin Use Case Diagram**

5 Non-Functional Requirements

Response Time:
Page load times under three seconds.

Concurrent Users:
Support for at least 500 simultaneous users.

Password Security:
User passwords must be stored using industry-standard cryptographic hashing (bcrypt or equivalent) with appropriate cost factors to prevent unauthorized access.

Data Preservation (Soft Deletes):
System must implement soft deletion for critical entities (users, orders, menu items) to maintain historical records and enable data recovery without permanent loss. Deleted records are marked with timestamps rather than physically removed from the database.

Continuous Integration/Deployment:
System must utilize automated CI/CD pipelines for testing, building, and deploying code changes to ensure consistent quality and rapid delivery across development, staging, and production environments.

API Security:
Rate limiting and input validation.

Backup & Recovery:
Automated daily backups with disaster recovery plan.

System Operating Hours (Food Service Providers):
7:00 AM - 5:00 PM, Saturday to Thursday. This allows providers to prepare before customer ordering begins and complete cleanup after customer hours end.

System Operating Hours (Food-truck Patrons):
8:00 AM - 4:40 PM, Saturday to Thursday.

6 Interface Requirements

Currently, no external integrations (e.g. credit card, external authentication).

7 Constraints

7.1 Technical Constraints

- **Web-Based Platform:** The system must be accessible via standard web browsers (Chrome, Firefox, Safari, Edge) without requiring native mobile applications in the initial release.

- **Technology Stack:** Development is limited to technologies and frameworks approved by GIU's IT infrastructure and familiar to the development team.

- **Database Management:** Must utilize a relational database system compatible with university hosting environments.

- **No External Payment Integration:** The initial version will not integrate third-party payment gateways; transactions will be handled offline or through existing university payment systems.

- **Campus Network Dependency:** Optimal performance is guaranteed only within GIU campus network infrastructure.

7.2 Operational Constraints

- **Operating Hours:** System functionality is limited to food truck operational hours (7:00 AM - 5:00 PM for providers, 8:00 AM - 4:40 PM for customers, Saturday to Thursday). The extended provider hours allow for preparation before service and cleanup after service.

- **Physical Pickup Required:** Orders must be physically collected from food trucks; no delivery service is provided.

- **Campus-Only Service:** The system serves only GIU campus food trucks and is geographically restricted to on-campus operations.

- **Manual Order Fulfillment:** Food preparation and handoff processes remain manual; the system only manages digital ordering and tracking.

7.3 Resource Constraints

- **Development Timeline:** The project must be completed within the academic semester timeframe as defined by the course requirements.

- **Team Size:** Development is limited to an 8-member student team with varying levels of experience.

- **Budget Limitations:** No dedicated budget for premium third-party services, APIs, or cloud infrastructure beyond what GIU provides.

- **Hardware Infrastructure:** Dependent on existing GIU server infrastructure and food truck operator devices (smartphones, tablets, or PCs).

7.4 Regulatory and Policy Constraints

- **Data Privacy Compliance:** Must adhere to GIU's data protection policies and student information privacy guidelines.
- **University Authentication:** Initial version will not integrate with GIU's central authentication system (planned for future releases).
- **Food Safety Standards:** The system does not enforce or monitor food safety regulations; responsibility remains with food truck operators.
- **Vendor Agreements:** Participation of food trucks is subject to approval and agreements with GIU administration.

7.5 User and Usability Constraints

- **Internet Connectivity:** Users must have stable internet connection to access the system; no offline functionality is provided.
- **Device Requirements:** Users need devices with modern web browsers; minimum screen size recommendations apply for optimal experience.
- **Digital Literacy:** Assumes basic computer and smartphone literacy among students, staff, and food truck operators.
- **Language Support:** Initial release supports English only; Arabic language support planned for future versions.

7.6 Design and Architectural Constraints

- **Scalability Scope:** System designed for GIU campus scale (approximately 500 concurrent users); not architected for multi-campus deployment in initial version.
- **Polling-Based Updates:** Order status updates operate on polling or periodic refresh mechanisms, not true real-time WebSocket connections.
- **Legacy System Integration:** No integration with existing GIU administrative systems (student information, financial systems) in the initial release.
- **Responsive Design Mandatory:** Must function across desktop, tablet, and mobile screen sizes despite being web-only.

8 Future Scope of the Project

Potential features for future versions:

- Integration with university authentication.
- Push notifications for order updates.
- Mobile app version.
- Payment gateway support.
- **Rating and Review System:** Allow customers to rate food trucks and menu items, providing feedback and helping others make informed choices.
- **Inventory Management:** Enable vendors to track stock levels, receive low-stock alerts, and automatically mark items as unavailable when ingredients run out.

Notes

Important Note

- This is a **living document**. It evolves with project development.
- At the end, this document may be retitled as "Project Documentation."

How the System Solves the Problem

The Challenge: Students and staff waste valuable time standing in long queues at food trucks during peak hours, with no visibility into wait times or order status.

Our Solution:

1. **Pre-ordering eliminates queues:** Users browse menus and place orders online before arriving at the truck, eliminating the need to wait in physical lines.
2. **Time visibility reduces uncertainty:** The system displays estimated preparation times and allows users to select pickup windows, helping them plan their schedule.
3. **Order tracking provides transparency:** Customers can check their order status (Pending → In Progress → Ready) through polling-based updates on the website, knowing exactly when to pick up their food.
4. **Busy mode prevents overload:** Vendors can activate busy mode when overwhelmed, preventing new orders and managing capacity effectively.
5. **Centralized platform streamlines operations:** Both customers and vendors access a single unified system, reducing confusion and improving coordination.

Result: Students and staff save time, avoid frustration, and enjoy a smoother campus dining experience, while food truck operators manage orders more efficiently and serve customers better.